

Tipos Abstractos de Datos (TAD)

Historia

- Datos primitivos
- '60 → Programación estructurada, Concepto de Tipos de datos y Operaciones
- '70 → Tipo Abstracto de datos
- ¿Qué es la abstracción?

Abstracción

La abstracción es la simplificación de un objeto o de un proceso de la realidad en la que sólo se consideran los aspectos más relevantes.

Tipo de Datos vs Estructura de datos

Tipo Abstracto de Datos

Un Tipo Abstracto de Datos es un modelo matemático con una serie de operaciones definidas en ese modelo.¹

Constituye una representación de una entidad real. Para definir un tipo de datos debemos definir el conjunto de valores y las operaciones que podemos realizar con estos valores. Los TAD son definidos mediante una especificación independiente de cualquier implementación.

Especificación / Implementación

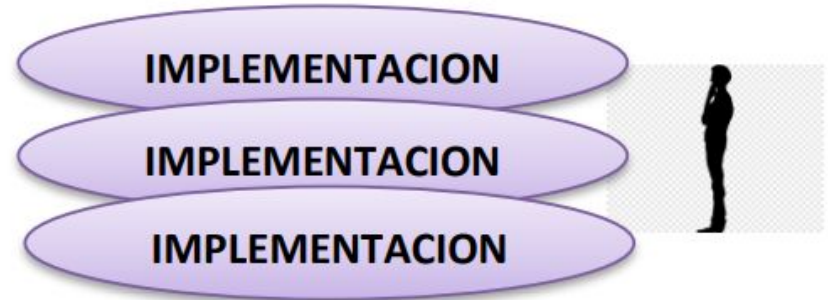
- Comportamiento - Qué
- Cantidad de implementaciones - Cómo
- ¿Qué ocurre cuando cambiamos la implementación de un TAD?

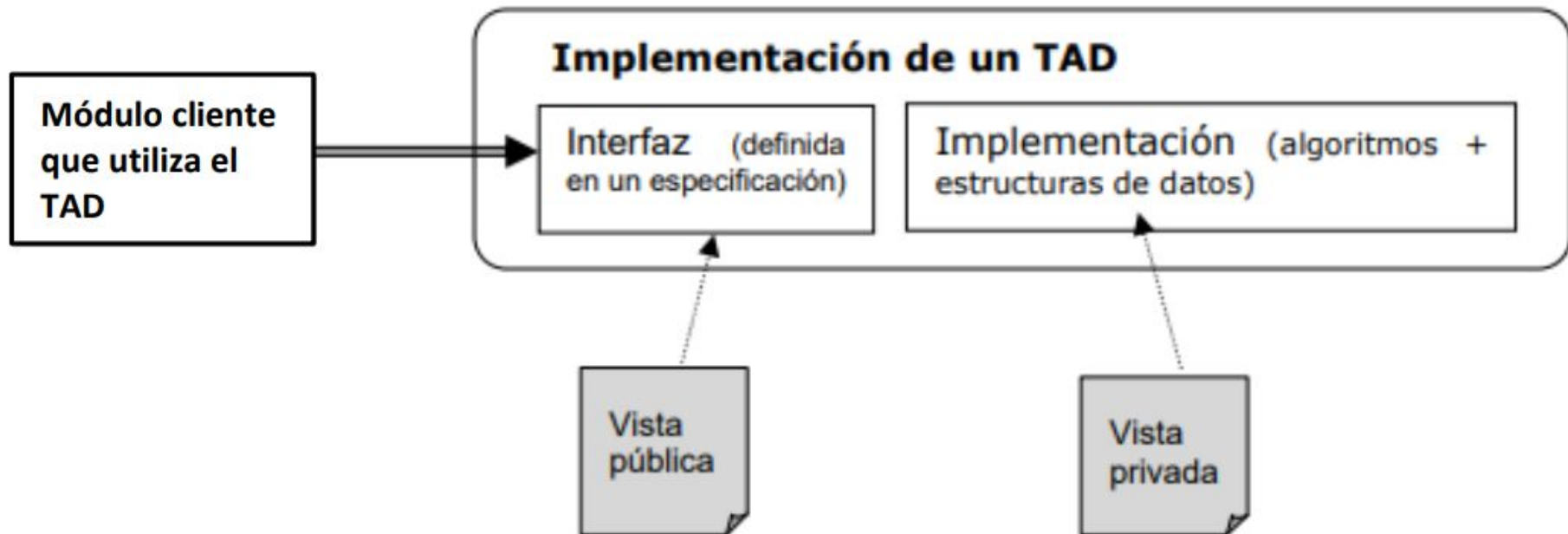
Propiedades de los TAD:

- Precisión
- Generalidad
- Legibilidad
- No-Ambigüedad

Características deseables

- Ocultamiento de información
- Encapsulamiento
- Reusabilidad
- Especificar comportamientos inesperados





¿Para qué sirve la abstracción?

Tipos de abstracción

- **Abstracción funcional**
 - Software modular y fiable
 - Diferenciar el qué del cómo
 - Ocultación de la información
- **Abstracción de datos**
 - Nuevos tipos de datos
 - Software corto, legible y flexible

Invariante de representación

Definición: Afirmación sobre un objeto que debe ser cierta en todo momento de su existencia.

- Si tenemos instancias que deben representar estados específicos, ¿En qué etapa trabajaremos más (en la especificación o en la implementación)?

Verificación del invariante.

Ejemplo 1

Especificar el TAD Naturales (modela los nros naturales incluyendo el 0).

Especificación:

Creamos la interfaz del TAD Nat definiendo distintas operaciones y valores.

Invariante de representación: $n \geq 0$

Crear Nat	Nat Crear (n)	Crea un nuevo Nat
Sumar 2 Nat	Nat Sumar (Nat1, Nat2)	Suma dos Nat y devuelve un Nat
Restar 2 Nat	Nat Restar (Nat1, Nat2)	Resta dos Nat y devuelve un Nat
es par el Nat?	Boolean esPar (Nat1)	Devuelve true si Nat1 es par

Ejemplo 2

A) Completar la interfaz del TAD Fecha (asumir que los días van de 1 a 31).

Invariante de representación:

$31 \geq \text{día} \geq 1$

$12 \geq \text{mes} \geq 1$

$9999 \geq \text{año} \geq 1$

Fecha(entero día, entero mes, entero año)	Constructor
Fecha sumar(Fecha f1, Fecha f2)	Suma dos fechas y devuelve una fecha con el resultado.
Entero dia(Fecha f)	Devuelve el día de la Fecha
Entero mes(Fecha f)	Devuelve el mes de la Fecha
Entero año(Fecha f)	Devuelve el año de la Fecha
Lógico esBisiesto(Fecha f)	Devuelve true si el año es bisiesto

Ejemplo 2

B) Completar la implementación del TAD Fecha según la especificación anterior.

```
public class Fecha {  
    private int dia;  
    private int mes;  
    private int año;
```

```
Fecha(int dia, int mes, int año){  
    if (dia>31){  
        throw new RuntimeException("El dia debe estar entre 1 y 31");  
    }  
    //...  
    this.dia = dia;  
    this.mes = mes;  
    //...  
}
```

```
public int dia(){  
    return dia;  
}
```

```
public int mes(){  
    return mes;  
}
```

```
public int año(){  
    return año;  
}
```

```
public void sumar(Fecha fecha){
```

```
    dia = dia + fecha.dia;
```

```
    if (dia > 31){
```

```
        mes = mes + 1;
```

```
        dia = dia - 31;
```

```
    }
```

```
    //...
```

```
}
```

@Override

```
public String toString(){  
    return dia + "/" + mes + "/" + año;  
}
```

Todas las clases de un TAD deben implementar el método toString y tener un test que compruebe algún caso de uso.


```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
    Fecha fecha1 = new Fecha(1,1,1);  
    Fecha fecha2 = new Fecha(31,1,1);  
    fecha1.sumar(fecha2);  
    System.out.println(fecha1.toString());  
}  
}
```

TAD paramétricos (generics)

Como parte de la especificación se puede agregar un parámetro formal que determina sobre qué tipos de elementos va a operar el TAD.

Los tipos genéricos proporcionan los medios para crear modelos generales.

Podríamos escribir un método genérico para ordenar un arreglo de objetos, y después invocar el método genérico con arreglos Integer, arreglos Double, arreglos String y así en lo sucesivo, para ordenar los elementos del arreglo.

Podríamos escribir una sola clase Pila genérica para manipular una pila de objetos, y después instanciar objetos Pila para una pila de objetos Integer, una pila de objetos Double, una pila de objetos String, y así en lo sucesivo.

Sintaxis

La sintaxis en Java para especificar un parámetro formal es indicar un tipo de dato genérico entre corchetes “<>”, al final del nombre de cada clase del TAD.

Los parámetros formales se denominan variables de tipo, porque el dominio de dichas variables se compone de tipos de Java como también de tipos generados por los usuarios.

Ejemplo 4

Queremos simular el comportamiento del arreglo estático de java “[]”, que no soporta Generics, por un TAD que soporte Generics.

TAD ArregloEstatico<T> //T es una variable de tipo

Notar que el TAD no es “ArregloEstatico”, sino “ArregloEstatico<T>”. El tipo de T se determina en tiempo de compilación.

Dominio(T) = TiposJava U TiposUsuario

```
public class ArregloEstatico<T> {  
    ArrayList<T> datos;  
  
    public ArregloEstatico(int tamano){  
        datos = new ArrayList<T>( );  
        for (int i=0; i< tamano; i++){  
            datos.add(null);  
        }  
    }  
}
```

```
public T leer(int indice){  
    if (indice < datos.size())  
        return datos.get(indice);  
    else  
        throw new RuntimeException("indice incorrecto");  
}
```

```
public void escribir(T valor, int indice){  
    if (indice < datos.size())  
        datos.set(indice, valor);  
    else  
        throw new RuntimeException("indice incorrecto");  
}  
}
```

```
public class Test {  
    public static void main(String[] args) {  
        ArregloEstatico<String> a1 = new ArregloEstatico<String>(5);  
        ArregloEstatico<Fecha> a2 = new ArregloEstatico<Fecha>(3);  
        a1.escribir("hola0", 2);  
        a1.escribir("hola1", 3);  
        a2.escribir(new Fecha("01/01/2016"), 2);  
        System.out.println(a1.leer(3));  
        System.out.println(a2.leer(2).fecha);  
    }  
}
```


Recorrer una estructura de datos en Java

Hasta ahora vimos que podíamos recorrer un TAD o una estructura de datos de la siguiente manera:

```
ArrayList<Integer> lista = new ArrayList();
```

```
lista.add(1);
```

```
lista.add(2);
```

```
for (int i=0; i < lista.size(); i++){
```

```
    System.out.println(lista.get(i));
```

```
}
```

Sin embargo, desde java 1.5 existe una manera más eficaz (siempre que sea posible)

```
for (Integer elem: lista){  
    System.out.println(elem);  
}
```

De hacer lo mismo sin depender de que conozcamos como es la estructura en especial (menor acoplamiento de código – ver Clase 03 Diseño).

Más adelante, cuando veamos objetos, veremos cómo hacer que un TAD soporte esta sintaxis.

Actividad en clase

TAD Dirección

- a) Completar la interfaz del TAD Dirección.
- b) Completar la implementación del TAD Dirección según la especificación anterior.